Exploratory Data Analysis using R & RStudio

Dr. Rohit Vishal Kumar
Associate Professor (Marketing)
International Management Institute Bhubaneswar
Email: rvkumar@imibh.edu.in

September 2, 2021

Contents

1 Introduction to EDA **2**

2 Preparing RStudio for EDA **2**

3 Using ggplot2 **3**
3.1 Begining Charting 4
3.2 Using Facets 5
3.3 Using Multiple ggplot2 layers 6
3.4 Changing ggplot2 themes 8

4 Doing Exploratory Data Analysis **9**
4.1 Analysis of a Single Variable 10
4.1.1 Single Categorical Variable 10
4.1.2 Single Continuous Variable 11
4.2 Analysing Two Variables 13
4.2.1 Scatter Plots 14
4.2.2 Multiple Box Plot 15
4.2.3 Correlogram and Heat-map 15

5 Dealing with Missing Data **18**
5.1 Deletion: 19
5.2 Imputation Methods: 19
5.3 Prediction Model: 19
5.4 KNN Imputation: 19
5.5 Using R to deal with Missing Data 20

6 Summary **23**

7 References **23**

Please cite this document as follows:

Kumar, R. V. (2021). Exploratory Data Analysis using R & RStudio: pp. 1-23. Bhubaneswar, India: International Management Institute.

1 Introduction to EDA

Exploratory Data Analysis (EDA) (Tukey, 1997) can be simply defined as the critical process of performing initial investigations on data so as to discover patterns,to spot anomalies,to test hypothesis and to check assumptions with the help of summary statistics and graphical representations.

In statistics, exploratory data analysis is an approach of analyzing data sets to summarize their main characteristics, often using statistical graphics and other data visualization methods. A statistical model can be used or not, but primarily EDA is for seeing what the data can tell us beyond the formal modeling or hypothesis testing task. Exploratory data analysis was promoted by John Tukey to encourage statisticians to explore the data, and possibly formulate hypotheses that could lead to new data collection and experiments. EDA is different from initial data analysis (IDA), which focuses more narrowly on checking assumptions required for model fitting and hypothesis testing, and handling missing values and making transformations of variables as needed. EDA is also different from Statistical Graphing in the sense that the latter is mainly done for presentation and publication purposes; even though EDA is done using many techqniques of statistical graphing.

2 Preparing RStudio for EDA

In order to do EDA in R (R Core Team, 2021) and RStudio (RStudio Team, 2020) we require the installation of the `tidyverse` package which can be installed on as follows:

```
install.packages("tidyverse", dependencies = TRUE)
```

The rationale for choosing the package `tidyverse` (Wickham et al., 2019) is that the package contains the required package `ggplot2` which is necessary for doing exploratory data analysis. We shall be using dataset `mpg` — which is available in the `ggplot2` package — for understanding the exploratory data analysis. Alternatively, if so desired, users can directly load the package `ggplot2` using the above `install.package()` command by replacing `tidyverse` by `ggplot2`. The next step would be to load the library `tidyverse` and make all its functionality available to R engine. This is done by the `library()` command as follows:

```
library(tidyverse)

## Warning:  package 'tidyverse' was built under R version 4.1.1
## -- Attaching packages --------------------------------------- tidyverse 1.3.1 --
## v ggplot2 3.3.5     v purrr   0.3.4
## v tibble  3.1.4     v dplyr   1.0.7
## v tidyr   1.1.3     v stringr 1.4.0
## v readr   2.0.1     v forcats 0.5.1
## Warning:  package 'tibble' was built under R version 4.1.1
## Warning:  package 'readr' was built under R version 4.1.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
```

As you can see, the moment we attach the library `tidyverse` it goes on to load the 8 distinct package and makes it available to the *R* engine. We are not actually going to use the remaining packages but these packages makes it possible to read in almost any kind of data, clean it and arrange it for analysis with *R*. Our key focus would be on the `ggplot2` (Wickham, 2016) package which would be used to create the graphs and plots necessary for understanding the Exploratory Data Analysis. Also if you get, *Conflicts* — output do not worry about them. When we load the `tidyverse` environment, it also loads other packages and some function calls

conflict with other packages. No need to worry, as *R* masks these functions so that there is no conflict between the functions.

3 Using ggplot2

ggplot2 graphs are based on the phlosophy of data visualization. In most cases we start with the function `ggplot()`, supply a dataset and aesthetic mapping with function *aes*(). We then add on layers like `geom_point()` or `geom_histogram()`, scales like `scale_colour_brewer()`, faceting specifications like `facet_wrap()` and coordinate systems like `coord_flip()`. All the layers are additive in nature and are added on with the `+` sign. The only mandatory requirement is that it should begin with `ggplot()` and a data-set. In order to understand the functioning of the `ggplot()` we shall use the data-set `mpg`. The data-set `mpg` gets attached to R and can be seen by typing the following command:

```
mpg

## # A tibble: 234 x 11
##    manufacturer model      displ  year   cyl trans drv     cty   hwy fl    class
##    <chr>        <chr>      <dbl> <int> <int> <chr> <chr> <int> <int> <chr> <chr>
##  1 audi         a4           1.8  1999     4 auto~ f        18    29 p     comp~
##  2 audi         a4           1.8  1999     4 manu~ f        21    29 p     comp~
##  3 audi         a4           2    2008     4 manu~ f        20    31 p     comp~
##  4 audi         a4           2    2008     4 auto~ f        21    30 p     comp~
##  5 audi         a4           2.8  1999     6 auto~ f        16    26 p     comp~
##  6 audi         a4           2.8  1999     6 manu~ f        18    26 p     comp~
##  7 audi         a4           3.1  2008     6 auto~ f        18    27 p     comp~
##  8 audi         a4 quattro   1.8  1999     4 manu~ 4        18    26 p     comp~
##  9 audi         a4 quattro   1.8  1999     4 auto~ 4        16    25 p     comp~
## 10 audi         a4 quattro   2    2008     4 manu~ 4        20    28 p     comp~
## # ... with 224 more rows
```

To better understand the **structure** of the dataset we use the `str()` function as follows:

```
str(mpg)

## tibble [234 x 11] (S3: tbl_df/tbl/data.frame)
##  $ manufacturer: chr [1:234] "audi" "audi" "audi" "audi" ...
##  $ model       : chr [1:234] "a4" "a4" "a4" "a4" ...
##  $ displ       : num [1:234] 1.8 1.8 2 2 2.8 2.8 3.1 1.8 1.8 2 ...
##  $ year        : int [1:234] 1999 1999 2008 2008 1999 1999 2008 1999 1999 2008 ...
##  $ cyl         : int [1:234] 4 4 4 4 6 6 6 4 4 4 ...
##  $ trans       : chr [1:234] "auto(l5)" "manual(m5)" "manual(m6)" "auto(av)" ...
##  $ drv         : chr [1:234] "f" "f" "f" "f" ...
##  $ cty         : int [1:234] 18 21 20 21 16 18 18 18 16 20 ...
##  $ hwy         : int [1:234] 29 29 31 30 26 26 27 26 25 28 ...
##  $ fl          : chr [1:234] "p" "p" "p" "p" ...
##  $ class       : chr [1:234] "compact" "compact" "compact" "compact" ...
```

The first row us that it is a `tibble` with 234 rows (or cases) and 11 columns (or variables). A `tibble`, is a modern form of `data.frame`. Tibbles are `data.frames` that are lazy and surly: they do less (i.e. they dont change variable names or types, and dont do partial matching) and complain more (e.g. when a variable does not exist). This forces you to confront problems earlier, typically leading to cleaner, more expressive code. The next lines display the names of the variables, their type and some typical values. In our case we have `char` (stands for "character"), `num` (stands for "number") and `int` (stands for "integer"). There are other data types which we will discover subsequently. To read up on the structure of the dataset type the code below. This code starts the help server and displays information about the data set. Make a note of the proper names of the variables

3.1 Begining Charting

Once we are familiar with the variable names and the structure of the data, let us now start learning how to plot using `ggplot2`. Type the following line and R will come up with the following output:

```
ggplot(data=mpg) + geom_point(mapping = aes(x = displ, y = hwy))
```

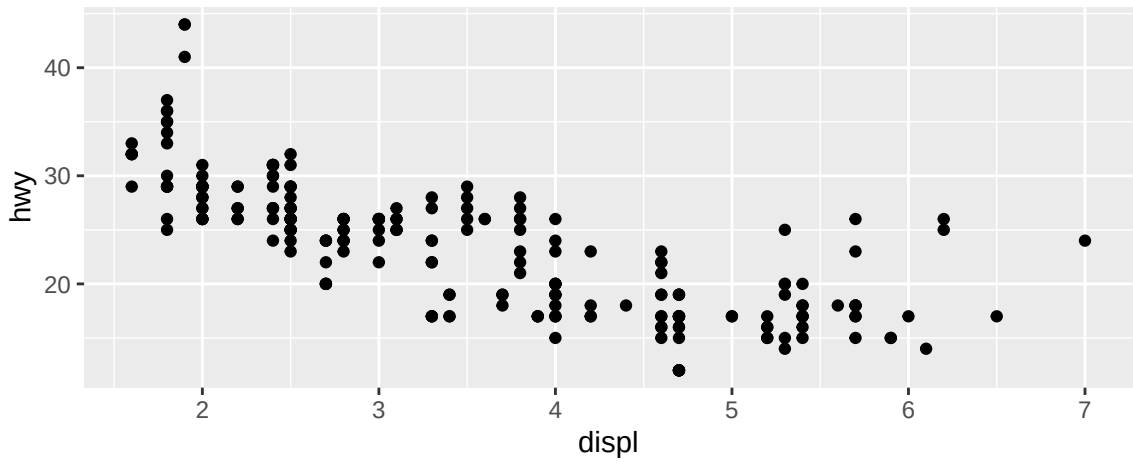

Figure 1: A simple ggplot2 output

As we already know `ggplot()` charts are made by adding layers. `ggplot(data = mpg)` is the base layer of `ggplot2` where we specify the data to be used for plotting purpose. `geom_point()` tells `ggplot2` to add a layer of points to the chart to create a scatterplot. The set of arguments inside the geom_points `(mapping = aes(x = displ, y = hwy))` tells which variables to plot on the x axis and y axis. As we can see, on the x-axis we have asked `ggplot()` to plot the "**displ**" (engine displacement) and on the y-axis we have asked it to plot the "**hwy**" (highway mileage). And that is exactly what `ggplot()` has plotted for us. The mapping argument is always paired with `aes()` — which is a shorthand for aesthetics. We can see from the figure that as the displacement increases the mileage tends to decrease. The problem with the above chart is that black points make it rather difficult to see. Would it be possible to add color to the chart. Well we can do that by adding the color outside of the `aes()` function as follows:

```
ggplot(data=mpg) + geom_point(mapping = aes(x = displ, y = hwy), color = "blue")
```

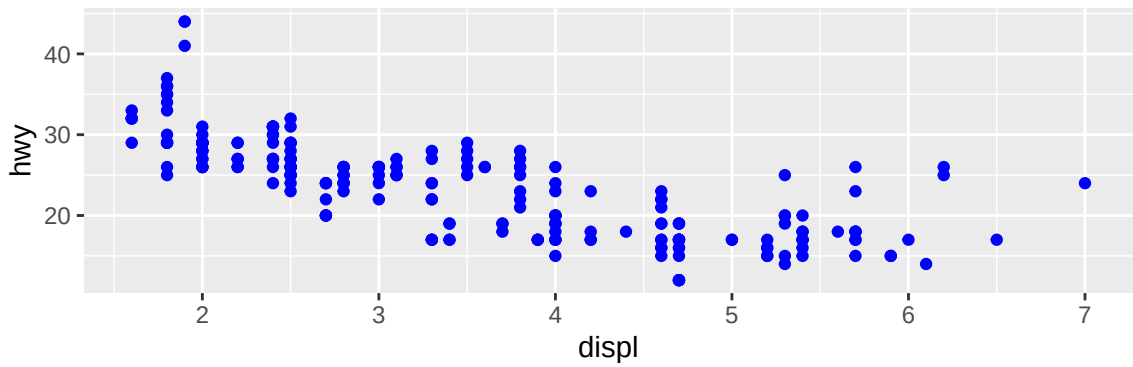

Figure 2: A `ggplot2` output with blue points

Not quite colorful but for the time this will do. Now looking at the chart we see that some

cars (situated on the right hand side) have huge displacements but also good highway mileage. Are these cars of some different class? Let's try to answer this question by coloring the chart with the `class` attribute. **We will allow `ggplot()` to handle the coloring and therefore this command would be different from the previous command in the sense that the `color by class` request would be passed on inside the `aes()` function.** This is essentially to tell `ggplot2` that we want the chart colored by a nominal variable and that it should decide how it will handle the coloring. Here is the code:

```
ggplot(data=mpg) + geom_point(mapping = aes(x = displ, y = hwy, color = class))
```

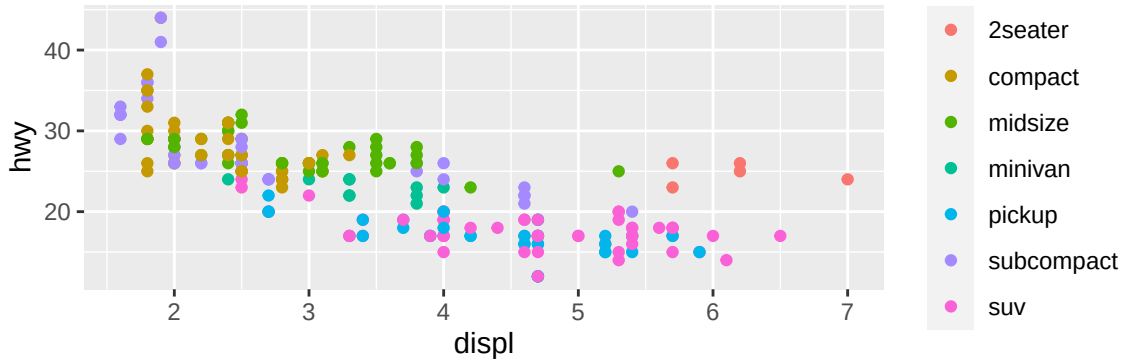

Figure 3: A `ggplot2` with points colored by Class

If you zoom in to the chart (figure 3) you will find that the dots which were causing us problems belong to the `2 seater` category. These are usually sports cars which have large engine displacements but due to the lower passenger loads give better mileage on the highway.

3.2 Using Facets

Facets are used to split the chart on the basis of a "**factor**". *Factor in R is defined a variable used to categorize and store the data, having a limited number of different values.* It stores the data as a vector of integer values. Factor in R is also known as a categorical variable that stores both string and integer data values as levels. Factor is mostly used in Statistical Modeling and Exploratory Data Analysis with R. Lets add a new layer using facet.

```
ggplot(data=mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, color = class)) +
  facet_wrap(~ class, nrow = 2)
```

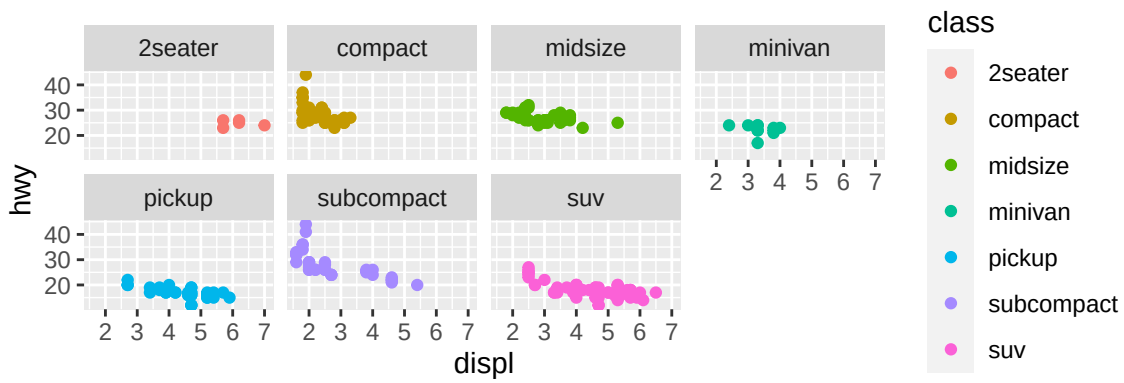

Figure 4: A Simple Facet Example

The sign ~ is read as 'by'; thus the `facet_wrap(~ class, nrow = 2)` tells `ggplot()` that we want the graph but this time we want to split up the graph into two rows (`nrow = 2`) on the basis of `class`. Now suppose we want to create slices by two different facets - the number of drive train (`drv`) and the number of cylinder (`cyl`) then we proceed as follows:

```
ggplot(data=mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, color = class)) +
  facet_grid( drv ~ cyl)
```

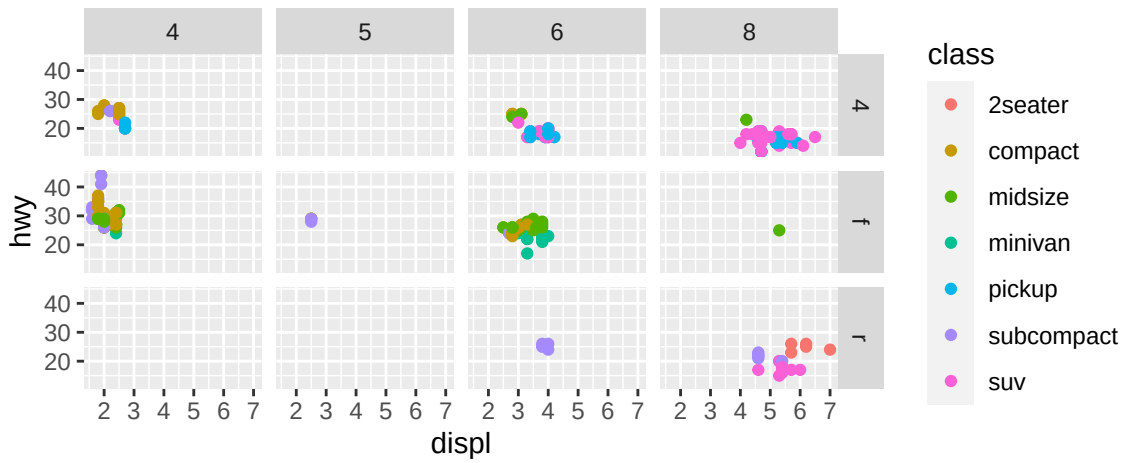

Figure 5: Faceting by two categorical variables

This chart covers a lot of grounds. On th extreme right it gives us the drive train types (`f` = front wheel drive, `r` = rear wheel drive, `4wd` = Four Wheel Drive). On the extreme top it gives us the number of cylinders (4,5,6 and 8). Each individual subplot shows the plot of `displ` (displacement) on the x-axis and of `hwy` (highway mileage) on the y-axis and the points are colored by class. In case we want to facet by only one variable we can use a dot (`'.'`) to represent the second variable. For example

```
ggplot(data=mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, color = class)) +
  facet_grid( drv ~ .)
```

Try the above chunk of code. It should provide a similar graph as the previous one but the number of cylinders on the top would be missing as we have only requested that the faceting be done using `drv`. Try replacing `facet_grid(drv ~ .)` by the following `facet_grid(. ~ cyl)` and check the results.

3.3 Using Multiple `ggplot2` layers

By now it should be clear as to how we can use the `ggplot2` function to create graphs. Let us now clarify a few more things and then move on to Exploratory Data Analysis. As of now we have only supplied the aesthetic mapping only to the added layers; but if we add the aesthetics to the `ggplot` layer we have a **global option** which will be used by any of the layers below the topmost layer. The global option can be over written by adding the desired aesthetics to the secondary plotting layers. For example, the command:

```
ggplot(data=mpg, mapping = aes(x = displ, y = hwy)) +
  geom_point()
```

will produce a chart which looks identical to the figure 1 on page 4. Good enough, but can we add another layer to the plot. The answer is yes and we will add a smoother layer to the chart as follows:

```
ggplot(data = mpg, mapping = aes(x = displ, y = hwy)) +
  geom_point() +  ## This is Layer 1 with the points
  geom_smooth()   ## This is layer 2 adding the smoother

## `geom_smooth()` using method = 'loess' and formula 'y ~ x'
```

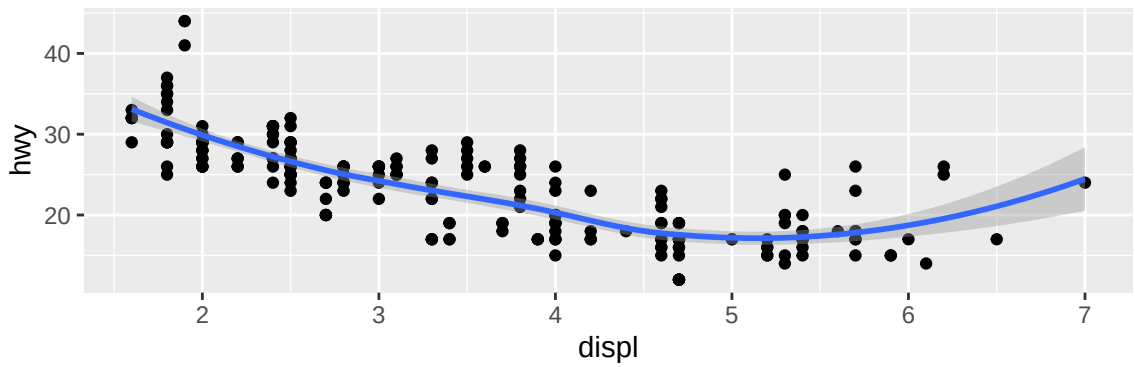

Figure 6: Adding Multiple Layers

The `geom_smooth()` function adds a `loess` smoother for a small dataset and for large dataset it adds a `gam` function. The function, by default, adds the standard error as it makes it easier to identify the trend. Replace `geom_smooth()` to `geom_smooth(se = FALSE)` and see the output. Let us plot another graph and see the results:

```
ggplot(data = mpg, mapping = aes(x = displ, y = hwy, color = drv)) +
  geom_point() + geom_smooth()

## `geom_smooth()` using method = 'loess' and formula 'y ~ x'
```

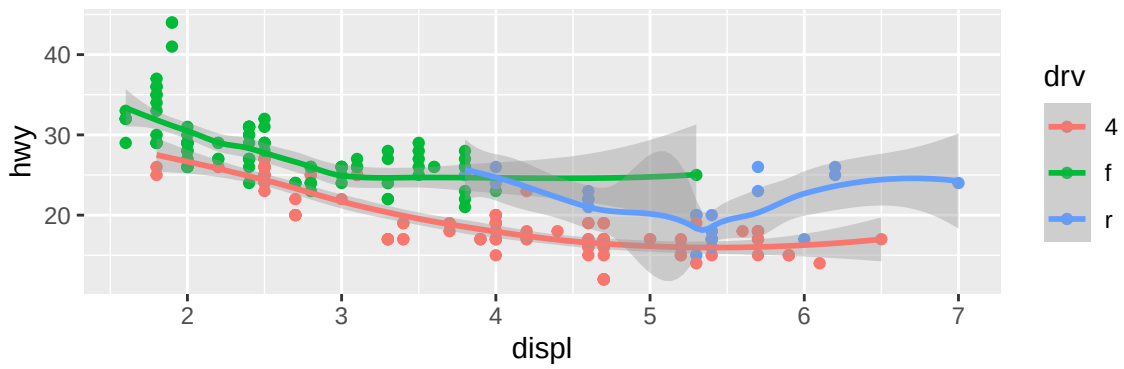

Figure 7: Adding Smoothers based on Drive Type

In figure 7 we have plotted the points and have added a smoother; but we have also instructed the `ggplot2` to add the layers by the drive type (`drv`) leading to `ggplot2` adding 3 smoother based on the drive type.

3.4 Changing ggplot2 themes

ggplot2 allows the expert user to do a lot of customization to the charts drawn. We shall not go into how we can control the individual elements; but rather on changing the appearances of the output. The theme that is used by default in ggplot2 is the theme_gray(); but here we shall see the use of two built in themes – theme_classic() and theme_void(). As R is an object oriented language, we can completely assign a series of command to a single object.

```
GPLOT <- ggplot(data=mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, color = class))
```

Here we have assigned the entire command to a new object GPLOT. Now, we will add the theme layer and see how it works

```
GPLOT + theme_void()
```

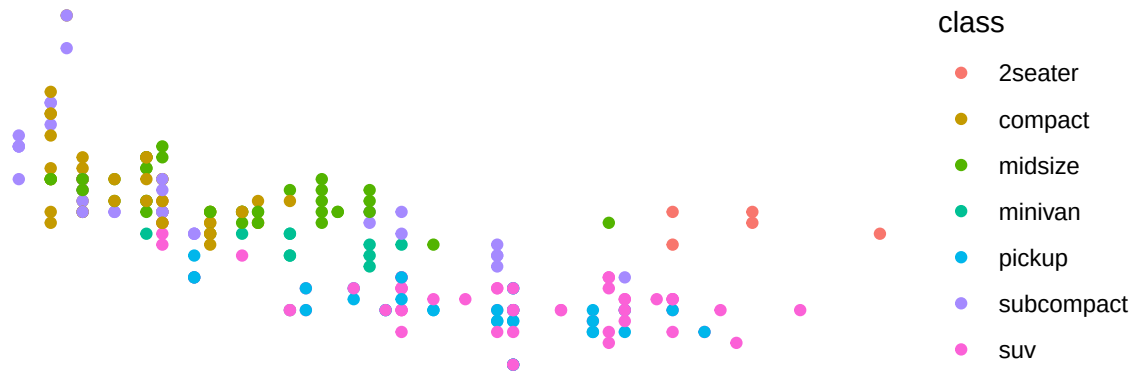

Figure 8: Using Theme Void

```
GPLOT + theme_classic()
```

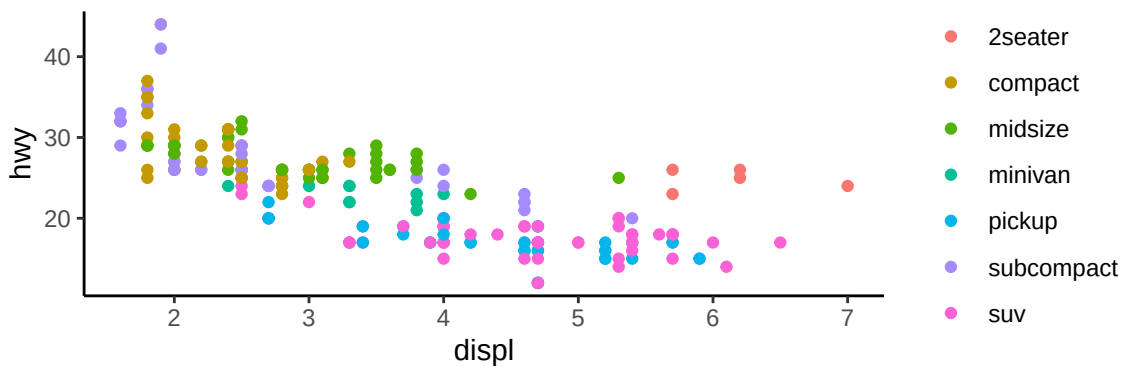

Figure 9: Using Theme Classic

The advantage of putting a lot of commands in another object is that it saves us a lot of typing. If the procedure confuses you then you should stick to the copy + paste + modify method of working. Secondly, the ggplot2 package gives you the following themes. Please feel free to try these themes at your leisure:

Theme	Description
theme_gray()	The default theme with grey background and white grid lines
theme_bw()	The classic dark-on-light `ggplot2` theme
theme_linedraw()	A theme with only black lines of various widths on white backgrounds
theme_light()	A theme similar to theme_linedraw() but with light grey lines and axes
theme_dark()	The dark cousin of theme_light() but with a dark background
theme_minimal()	A minimalistic theme with no background annotations
theme_classic()	A classic looking theme, with x and y axis lines and no grid lines
theme_void()	A completely empty theme

Table 1: Themes in `ggplot2`

4 Doing Exploratory Data Analysis

Exploratory Data Analysis (EDA) is an approach/philosophy for data analysis that employs a variety of techniques (mostly graphical) to (a) maximize insight into a data set (b) uncover underlying structure of the variables (c) extract important variables (d) detect outliers and anomalies (e) test underlying assumptions (f) develop parsimonious models and determine optimal factor settings. The EDA approach is precisely that – an approach – not a set of techniques, but an attitude/ philosophy about how a data analysis should be carried out. EDA is not identical to statistical graphics although the two terms are used almost interchangeably. Statistical graphics is a collection of techniques – all graphically based and all focusing on one data characterization aspect. EDA encompasses a larger venue; EDA is an approach to data analysis that postpones the usual assumptions about what kind of model the data follow with the more direct approach of allowing the data itself to reveal its underlying structure and model. EDA is not a mere collection of techniques; EDA is a philosophy as to how we dissect a data set; what we look for; how we look; and how we interpret. It is true that EDA heavily uses the collection of techniques that we call "statistical graphics", but it is not identical to statistical graphics per se.

Most EDA techniques are graphical in nature with a few quantitative techniques. The reason for the heavy reliance on graphics is that by its very nature the main role of EDA is to open-mindedly explore, and graphics gives the analysts unparalleled power to do so, enticing the data to reveal its structural secrets, and being always ready to gain some new, often unsuspected, insight into the data. In combination with the natural pattern-recognition capabilities that we all possess, graphics provides, of course, unparalleled power to carry this out.

The particular graphical techniques employed in EDA are often quite simple, consisting of various techniques of:

- Plotting the raw data (such as data traces, histograms, bihistograms, probability plots, lag plots, block plots, and Youden plots.
- Plotting simple statistics such as mean plots, standard deviation plots, box plots, and main effects plots of the raw data.
- Positioning such plots so as to maximize our natural pattern-recognition abilities, such as using multiple plots per page.

In this section we shall explore some of the techniques of EDA using the `mpg` dataset. We see from the structure on page 3 that the `mpg` dataset has a total of 234 rows and 11 columns. Out of these there are six variables which are character variables (alternatively known as discrete or categorical) and these are *manufacturer, model, trans, drv, fl and class.* `displ` is a numeric variable and `year, cyl, cty, hwy` are numeric integer variables.

4.1 Analysis of a Single Variable

How you visualize a variable would depend on whether the variable is categorical or continuous in nature. A variable is said to be categorical is it takes a small number of discrete values. In R, categorical variables are saved as "factors" or "character" variables and to examine them we use the **bar chart**.

4.1.1 Single Categorical Variable

We already know which of the variables are categorical in nature, so lets plot a bar chart:

```
ggplot(data = mpg) + geom_bar(mapping = aes(x = class))
```

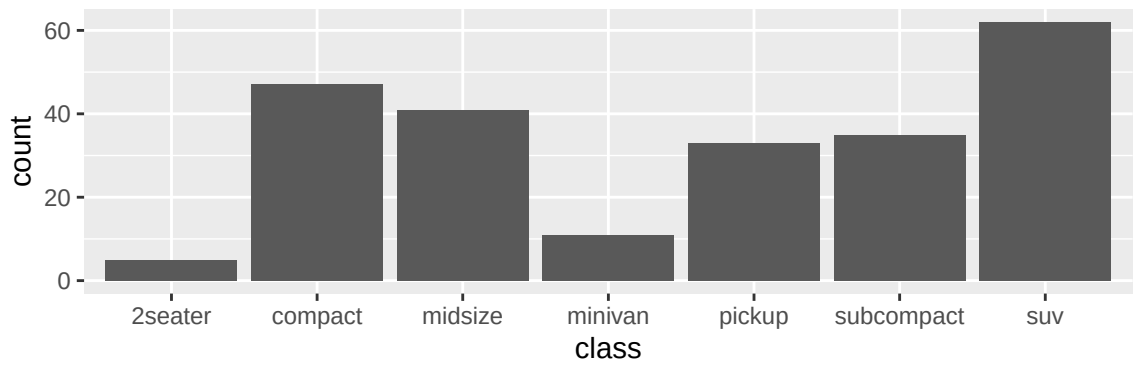

Figure 10: Vertical Bar Chart of Car Types

Okay fine. Now we want the graph to change orientation to horizontal.

```
ggplot(data = mpg) + geom_bar(mapping = aes(y = class))
```

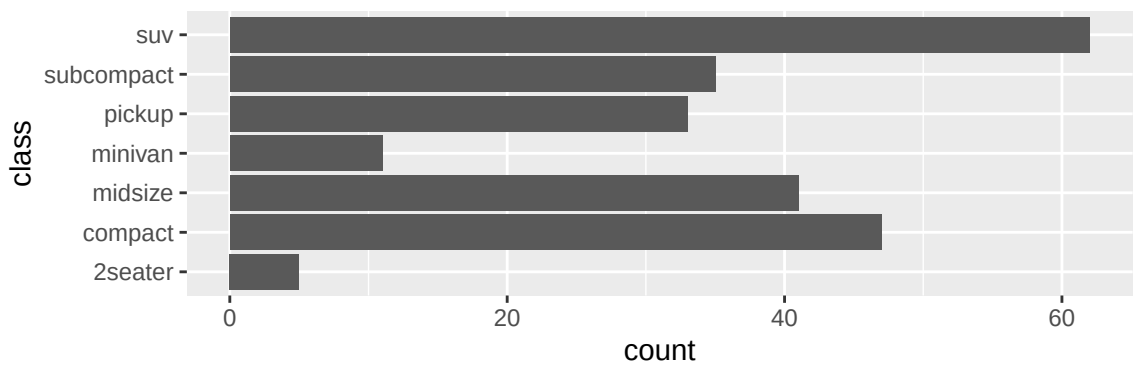

Figure 11: Horizontal Bar Chart of Car Types

Pretty good but the bar chart does not give us any major information. A question naturally comes is whether we can count the number of such cars. This requires a little trickery and the use of `dplyr()` package. If you have not loaded the `dplyr()` package please do so using the `library()` function. In case you loaded the `tidyverse()` package then the package should be loaded by default. In any case try the following command:

```
mpg %>% count(class)

## # A tibble: 7 x 2
```

```
##   class          n
##   <chr>      <int>
## 1 2seater        5
## 2 compact       47
## 3 midsize       41
## 4 minivan       11
## 5 pickup        33
## 6 subcompact    35
## 7 suv           62
```

The "%>%" is known as the **piping operator** and is used to redirect the output of one command to another command. The `count()` function is available in `dplyr()` package and the command effectively says that from the dataset `mpg` take the `class` variable and run it through the `count()` function and return the results. The result are illuminating – SUV tends to dominate the market, followed by midsized cars, followed compact cars, sub-compacts, pickups, minivans and two-seaters. Let us now go one step futher and ask `ggplot2` to add colour as per the drive.

```
ggplot(data = mpg) + geom_bar(mapping = aes(y = class, fill = drv))
```

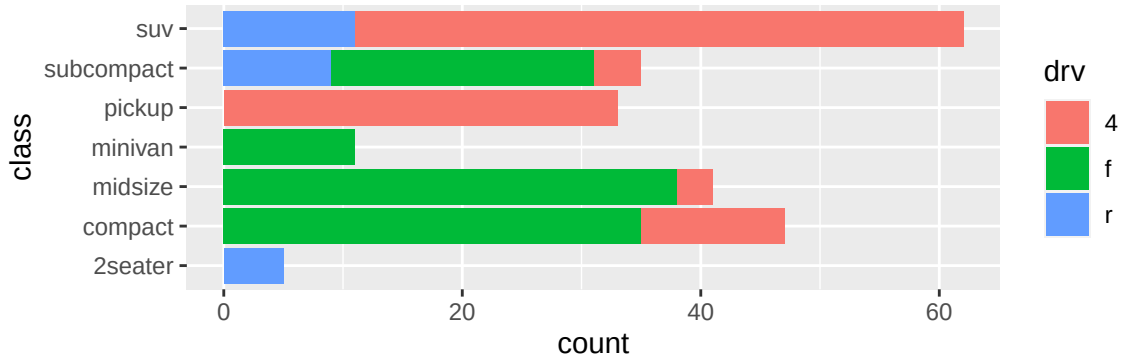

Figure 12: Horizontal Bar Chart Car Types filled by Drive Train

The results are even more interesting. 4-wheel drive are most common in SUV segment and pickup segment. The rear wheel drive are mostly seen in two-seater and the SUV segment; while other car types generally have front wheel drive.

4.1.2 Single Continuous Variable

In case of a continuous variable we have a larger set of analysis available. First and foremost we could create a histogram to check the distribution of the continuous variable as follows:

Now the above set of commands do a lot of functions. Let us examine them one by one. `geom_histogram()` tells `ggplot()` to draw the histogram of the city mileage. The `bins = 10` inform `ggplot()` that divide the data into 10 equal class interval. Additionally, we have instructed it to use the color `grey` to fill the histogram. In the next layer, `geom_freqpoly()` is used to add the frequency polygon to the city mileage. The `binwidth = 3` is used to inform that we want to have the width of the bins as 3 and `color = "red"` informs the `ggplot()` to draw the frequency polygon in red color. You should note here that if you leave the default values for `bin` and `binwidth`, `ggplot()` may complain and you will have to use trial and error method to decide which number produces the best histogram and the frequency polygon. Lastly, to make the diagram more visible, I have instructed `ggplot()` to use the `theme_light()` to draw the diagram (see figure 13 on page 12).

```
ggplot(data = mpg, mapping = aes(x = cty)) +  geom_histogram(bins = 10, fill = "grey") +
  geom_freqpoly(binwidth = 3, color = "red") + theme_light()
```

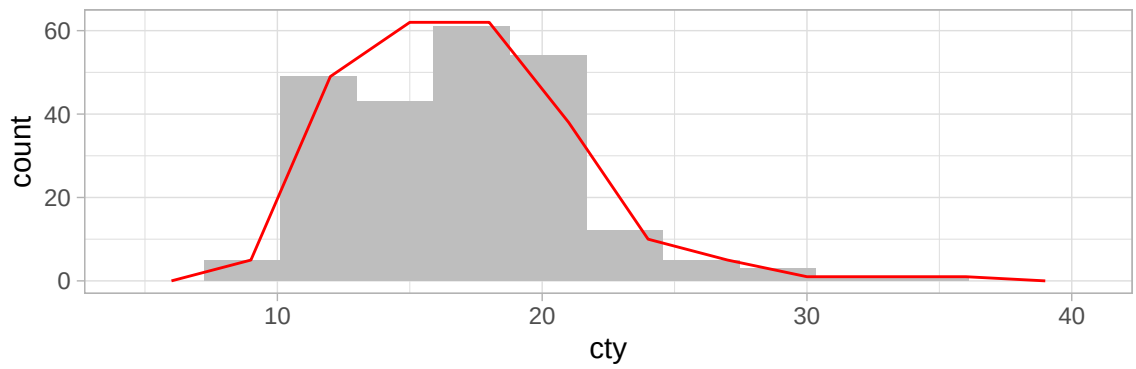

Figure 13: Histogram of City Mileage

The diagram tells us that the city mileage is slightly skewed to the right. To check whether it is really so, we run the summary command and get the following results:

```
summary(mpg$cty)

##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    9.00   14.00   17.00   16.86   19.00   35.00
```

The results confirm our belief from the histogram. The distance between 3rd quartile and maximum value is 16.00 where as the distance between 1st quartile and minimum is 5.00. To reconfirm the values let us run the same data and plot it into a boxplot.

```
ggplot(data = mpg, mapping = aes(x = cty)) + geom_boxplot()
```

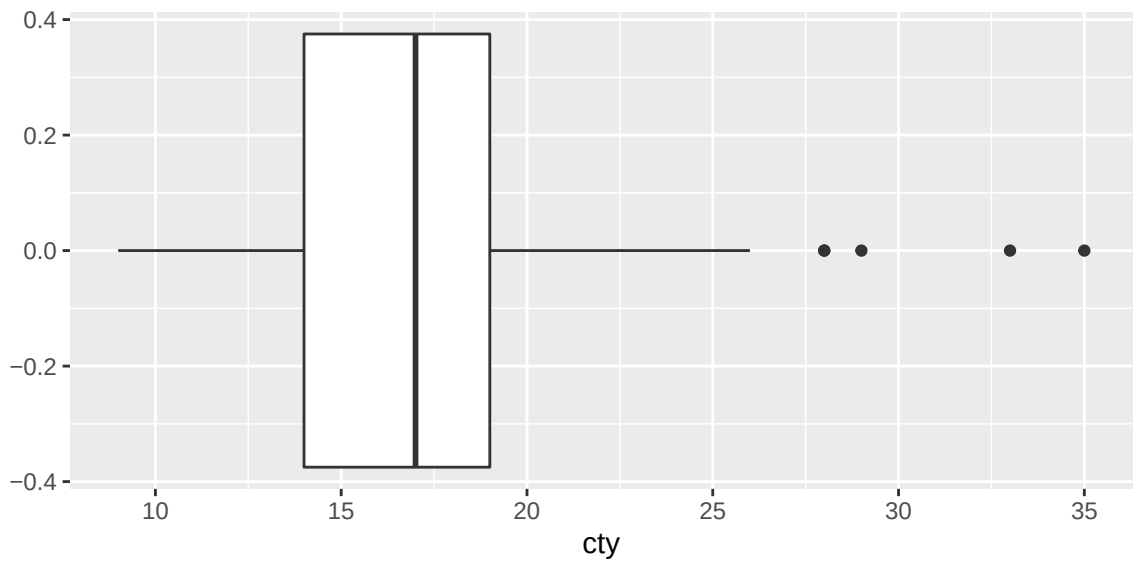

Figure 14: Boxplot of City Mileage

A **boxplot** is a type of visual shorthand for a distribution of values that is popular among statisticians. Each boxplot consists of a box that stretches from the 25th percentile (1st quartile) of the distribution to the 75th percentile (3rd quartile), a distance known as the interquartile

range (IQR). In the middle of the box is a line that displays the median, i.e. 50th percentile, of the distribution. These three lines give you a sense of the spread of the distribution and whether or not the distribution is symmetric about the median or skewed to one side. Visual points display observations that fall more than 1.5 times the IQR from either edge of the box. These outlying points are unusual so are plotted individually and are known as **outliers**. A line (or whisker) that extends from each end of the box and goes to the farthest non-outlier point in the distribution.

Another important plot in case of continuous variable is the QQ (Quantile-Quantile) plot – which is used to test whether the continuous variable is following a normal distribution or not. This is achieved by the following commands:

```
ggplot(data = mpg, mapping = aes(sample = cty)) +
  stat_qq(color = "red") + stat_qq_line(color="blue")
```

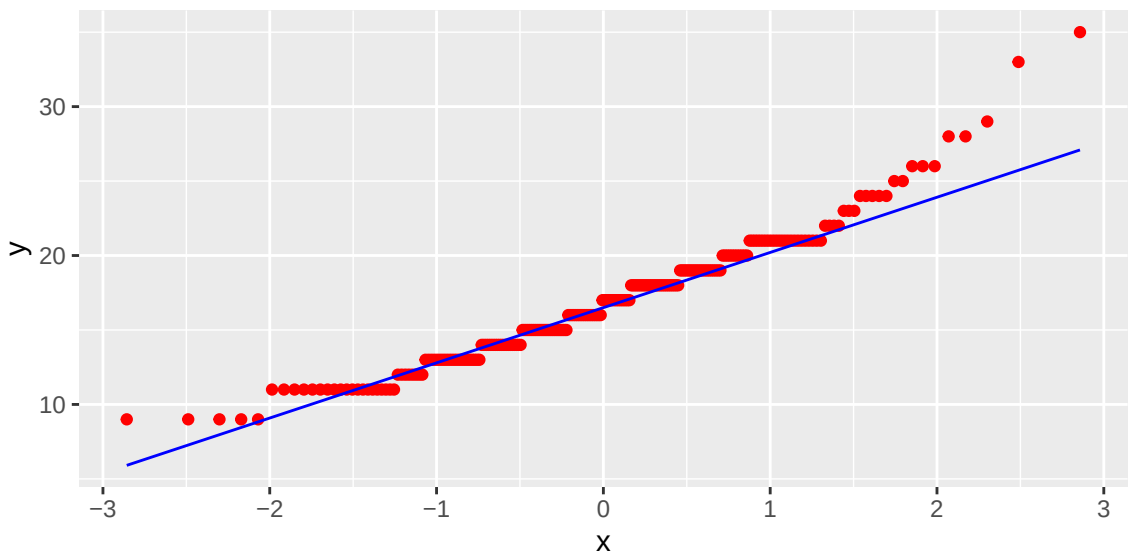

Figure 15: QQ Plot of City Mileage

The quantile quantile plot is an interesting plot but before explaining the QQ plot please note the difference in the `aes()` function of the plot. QQ plot takes in `(sample = variable)` as opposed to the `x = variable`. This difference is because on the x-axis the normal distribution is plotted and on the y-axis the data is plotted. The y-axis data is considered to be a sample and the quantiles are computed from the data using the `stat_qq()` function. For ease of visibility, I have requested `ggplot()` to plot the sample quantile in "red" color and the normal distribution line (`stat_qq_line()`) in "blue" color. For interpretation purpose, if the points are far away from the normal distribution line then the sample is non-normal and vice-versa. This is an eyeballing technique and more precise tests are available in R.

4.2 Analysing Two Variables

When it comes to analysing two variables the issues are a bit more complicated as there are be the following combination of variables – (a) categorical versus categorical (b) categorical versus continuous or (c) continuous versus continuous. To analyse these combinations require a bit more ingenuity than analyzing the single variable; but we shall restrict our analysis to simple plots which are easily doable.

4.2.1 Scatter Plots

One of the most used plot for data analysis is the scatter plot. Scatter plot help us understand the nature of relationship between two continuous variables. In order to make a scatter plot we use the same set of commands that were used to produce figure 1–3 on page 4 – 5 respectively. Lets make a new scatter plot between city and highway mileage:

```
ggplot(data = mpg, mapping = aes(x = cty, y = hwy)) + geom_point()
```

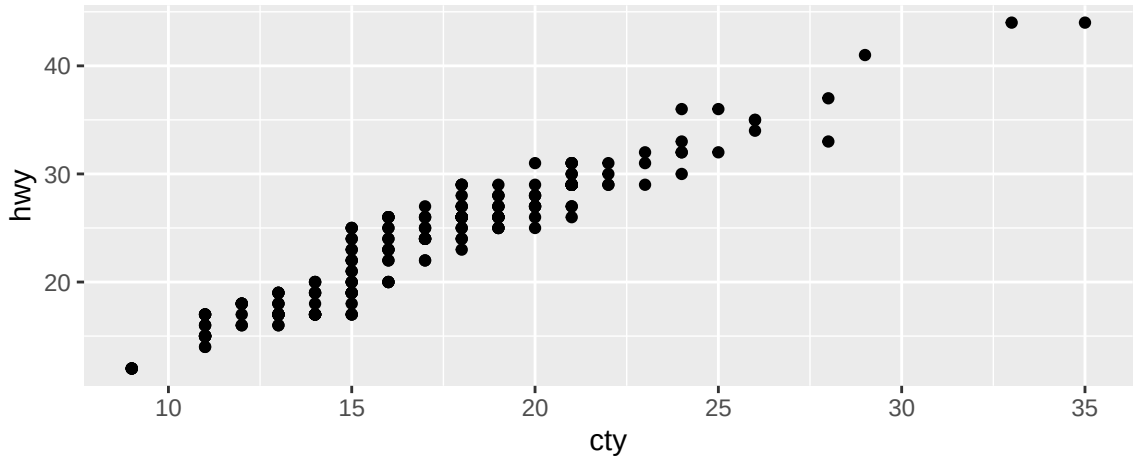

Figure 16: Scatter Plot of City Mileage versus Highway Mileage

Figure 16 tells us that city and highway mileage have a very strong positive relationship. The original data has 234 data points but the chart seems to display fewer points. This is because both `cty` and `hwy` are integers and there are many overlapping points appearing as a single dot. This can be handled by using `geom_jitter()`. The jitter function randomly jitters around original points based on a threshold controlled by the `width` and `size` argument. Lets also add a straight line smoother to the plot while we are at it:

```
ggplot(data = mpg, mapping = aes(x = cty, y = hwy)) + geom_point() +
  geom_smooth(method = "lm") + geom_jitter(width = 0.5, size = 1)

## `geom_smooth()` using formula 'y ~ x'
```

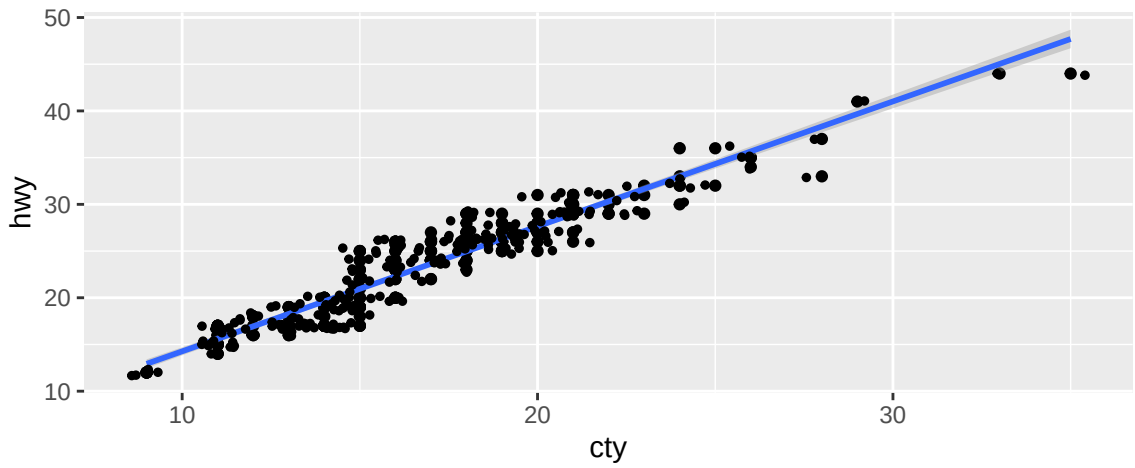

Figure 17: Scatter Plot with Jitter + Smoother Line of City & Highway Mileage

4.2.2 Multiple Box Plot

For two variables – one continuous and the other categorical – it sometimes makes more sense to use multiple bar charts. We have already seen how to create boxplot (refer figure 14 on page 12); we shall use the same to draw multiple boxplots but shall segregate the same on the basis of class:

```
ggplot(data = mpg, mapping = aes(x = cty, y= class)) + geom_boxplot(fill = "plum")
```

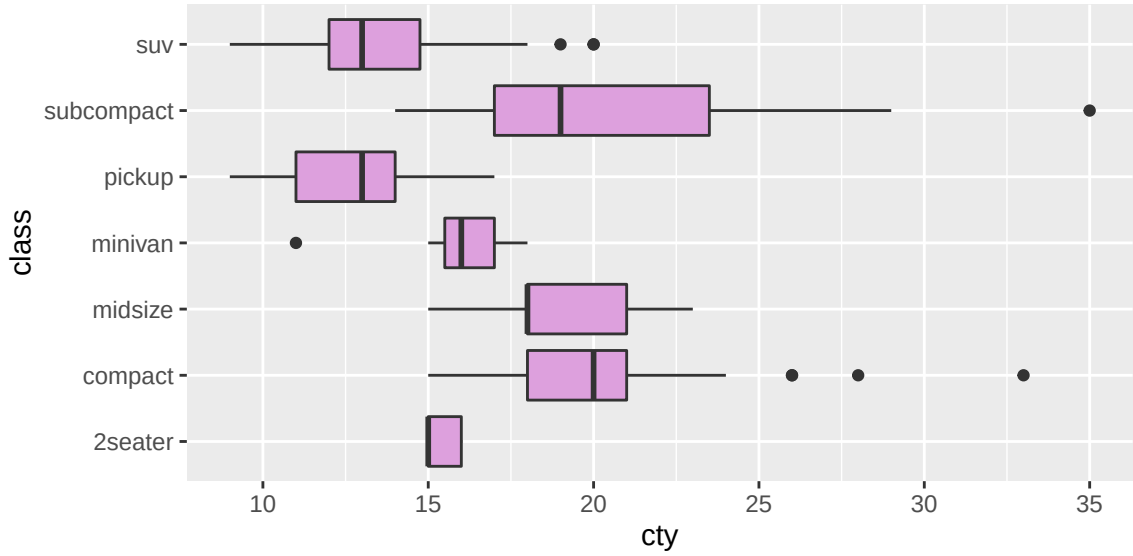

Figure 18: Boxplot of City Mileage by Class

Another way of looking at the data would be a violin plot in which the violin are effectively the histogram for the particular variable. It is created by adding `geom_violin()`. This exercise if left to the readers of the document. The analysis of boxplot shows up some interesting outputs. For example, the maximum variation in city mileage occurs in case of subcompacts and the minimum most variation occurs in terms of two seaters. The compact category, for example, provides about 15 miles per gallon to 23 miles per gallon; but has three outliers. SUV and pickups provide the lowest mileage, but there are two SUV which are outperforming other car categories in terms of city mileage. Another set of insight that is provided is about the skewness of the data. Subcompact mileage is highly skewed towards the right; which in effect is a good thing as the subcompact industry may be investing more in improving the mileage of the subcompacts. On the other hand, two seaters have one of the least city mileage in the category and no spread is seen in the category. This may lead one to infer that the two seater category does not sell on the basis of mileage but on other factors. Similar analysis can be done for highway mileage and some useful insights can be drawn into the data.

4.2.3 Correlogram and Heat-map

Another very important plot is the correlogram plot which lets us examine the correlation of multiple continuous variables present in the same dataset. This is conveniently implemented using the `ggcorrplot()` (Kassambara, 2019) package. As the dataset being used till now is mostly categorical I would be using the `diamonds` dataset. The dataset is available with the `ggplot()` library. First we shall load the library and the dataset:

```
install.packages("ggcorrplot") ## install the library ggcorrplot
library(tidyverse) ## load all packages and datasets
```

```r
library(ggcorrplot) ## load the library ggcorrplot
str(diamonds)  ## Check the structure of the diamonds dataset

## tibble [53,940 x 10] (S3: tbl_df/tbl/data.frame)
##  $ carat  : num [1:53940] 0.23 0.21 0.23 0.29 0.31 0.24 0.24 0.26 0.22 0.23 ...
##  $ cut    : Ord.factor w/ 5 levels "Fair"<"Good"<..: 5 4 2 4 2 3 3 3 1 3 ...
##  $ color  : Ord.factor w/ 7 levels "D"<"E"<"F"<"G"<..: 2 2 2 6 7 7 6 5 2 5 ...
##  $ clarity: Ord.factor w/ 8 levels "I1"<"SI2"<"SI1"<..: 2 3 5 4 2 6 7 3 4 5 ...
##  $ depth  : num [1:53940] 61.5 59.8 56.9 62.4 63.3 62.8 62.3 61.9 65.1 59.4 ...
##  $ table  : num [1:53940] 55 61 65 58 58 57 57 55 61 61 ...
##  $ price  : int [1:53940] 326 326 327 334 335 336 336 337 337 338 ...
##  $ x      : num [1:53940] 3.95 3.89 4.05 4.2 4.34 3.94 3.95 4.07 3.87 4 ...
##  $ y      : num [1:53940] 3.98 3.84 4.07 4.23 4.35 3.96 3.98 4.11 3.78 4.05 ...
##  $ z      : num [1:53940] 2.43 2.31 2.31 2.63 2.75 2.48 2.47 2.53 2.49 2.39 ...
```

Now from the `diamonds` package we see that the `carat`, `depth`, `table`, `price`, `x`, `y` and `z` are the numeric variables. However, we may wish to label some of the variables anew as because "x" is length, "y" is width and "z" is depth and the original variable "depth" is actually called depth percentage. To check these lets run the following:

```r
?diamonds
```

We also need to pull the numeric variables and discard the non numeric variables for correlation to be computed. We do this as follows:

```r
attach(diamonds) ## Attach the dataset into the  workspace
DIA <- cbind(price, carat, depth, table, x, y, z)
head(DIA)

##      price carat depth table    x    y    z
## [1,]   326  0.23  61.5    55 3.95 3.98 2.43
## [2,]   326  0.21  59.8    61 3.89 3.84 2.31
## [3,]   327  0.23  56.9    65 4.05 4.07 2.31
## [4,]   334  0.29  62.4    58 4.20 4.23 2.63
## [5,]   335  0.31  63.3    58 4.34 4.35 2.75
## [6,]   336  0.24  62.8    57 3.94 3.96 2.48
```

As can be seen the dataset DIA has the variables `x, y` and `z`. We shall now replace the entire column names as follows

```r
colnames(DIA) <- c("Price", "Carat", "DepthPct", "Table", "Length", "Width", "Depth")
head(DIA)

##      Price Carat DepthPct Table Length Width Depth
## [1,]   326  0.23     61.5    55   3.95  3.98  2.43
## [2,]   326  0.21     59.8    61   3.89  3.84  2.31
## [3,]   327  0.23     56.9    65   4.05  4.07  2.31
## [4,]   334  0.29     62.4    58   4.20  4.23  2.63
## [5,]   335  0.31     63.3    58   4.34  4.35  2.75
## [6,]   336  0.24     62.8    57   3.94  3.96  2.48
```

Okay now that we have got the variables in one place, it is time to compute the correlation matrix. The function `cor(DIA)` will compute the correlation matrix on all the variables in the data frame DIA. Let us compute the correlation and store it into a matrix named as CORMAT. While using the `cor()` the correlation matrix would be computed to about 7 decimal places; which makes reading the thing pretty exhausting. So we also apply the `round()` function and round up the results to 3 decimal places

```
CORMAT <- round(cor(DIA),3)
CORMAT

##           Price Carat DepthPct  Table Length  Width Depth
## Price     1.000 0.922   -0.011  0.127  0.884  0.865 0.861
## Carat     0.922 1.000    0.028  0.182  0.975  0.952 0.953
## DepthPct -0.011 0.028    1.000 -0.296 -0.025 -0.029 0.095
## Table     0.127 0.182   -0.296  1.000  0.195  0.184 0.151
## Length    0.884 0.975   -0.025  0.195  1.000  0.975 0.971
## Width     0.865 0.952   -0.029  0.184  0.975  1.000 0.952
## Depth     0.861 0.953    0.095  0.151  0.971  0.952 1.000
```

Now it is a simple matter to produce the **heatmap** as follows:

```
ggcorrplot(CORMAT)
```

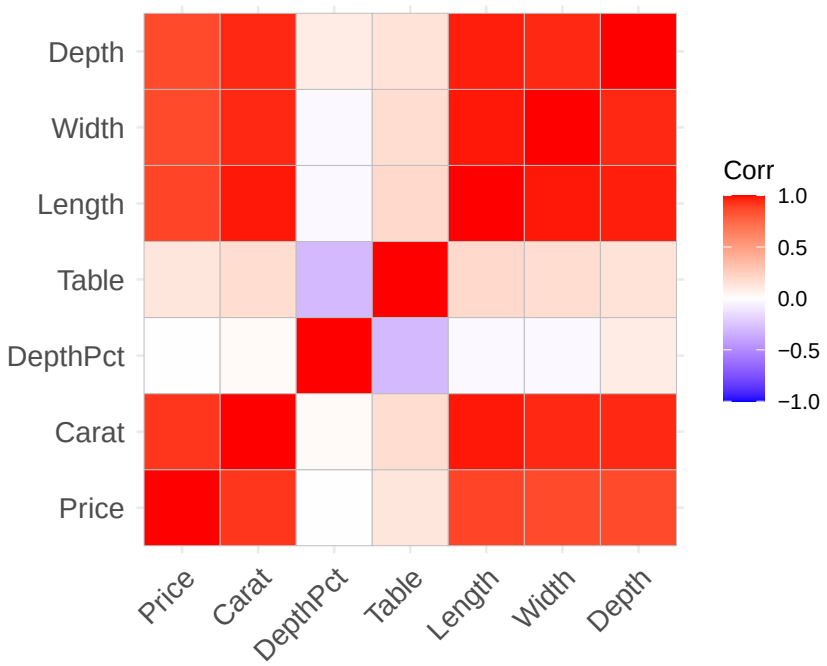

Figure 19: Heatmap of the Computed Correlation Matrix

A **heatmap** is a data visualization technique that shows magnitude of a phenomenon as color in two dimensions. The variation in color may be by hue or intensity, giving obvious visual cues to the reader about how the phenomenon is clustered or varies over space. There are two fundamentally different categories of heat maps: the **cluster heatmap** and the **spatial heatmap**. In a cluster heat map, magnitudes are laid out into a matrix of fixed cell size whose rows and columns are discrete phenomena and categories, and the sorting of rows and columns is intentional and somewhat arbitrary, with the goal of suggesting clusters or portraying them as discovered via statistical analysis. The size of the cell is arbitrary but large enough to be clearly visible. By contrast, the position of a magnitude in a **spatial heatmap** is forced by the location of the magnitude in that space, and there is no notion of cells; the phenomenon is considered to vary continuously.

The heatmap drawn in figure 19 is a **cluster heatmap** and uses three colors to reflect the correlation – `blue` for a negative correlation, `white` for zero correlation and `red` for positive correlation. However the problem with such a heatmap is that it is pretty difficult to figure out what each cell represents. In our case we have seven (7) variables and as such it is easy to see; but suppose we had 30 variables; in such a case it becomes difficult to understand what

is going in. Fortunately, `ggcorrplot()` provides a lot of customisations which can be used to make things easier. Let us run the following command and see the output:

```
ggcorrplot(CORMAT, method = "circle", type = "lower", ggtheme = ggplot2::theme_classic,
           lab = TRUE, lab_size = 3, colors = c("green", "white", "pink"))

## Warning:  `guides(<scale> = FALSE)` is deprecated.  Please use `guides(<scale> = "none")`
instead.
```

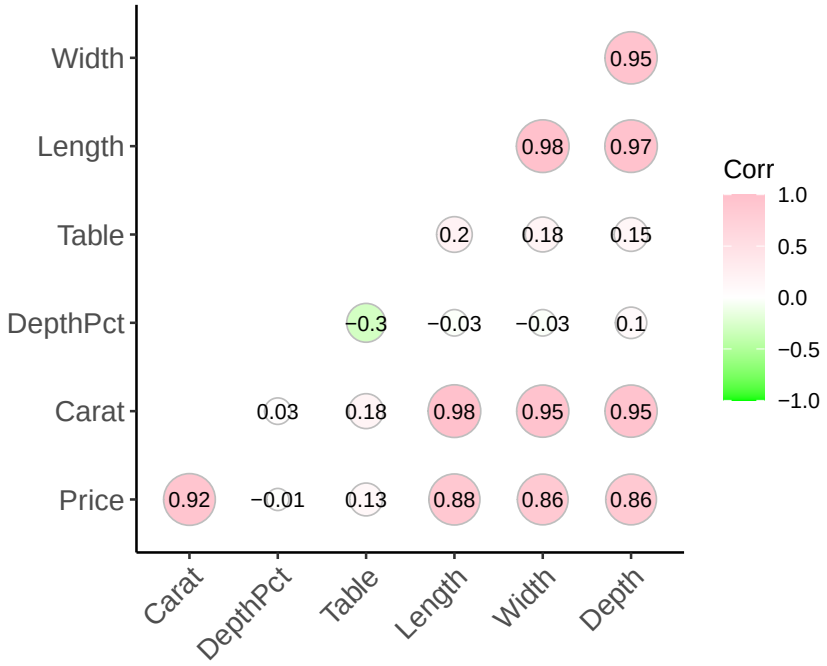

Figure 20: Lower Triange Heatmap with modifications

As usual `ggcorrplot()` allows us to apply a plethora of customization to the heatmap. For example `method =` can be either `square` or `circle`. The default is `square`. Similarly the `type =` can be either `full` (default) or `lower` or `upper`. `ggtheme =` can be used to apply any of the theme provided by the `ggplot2()` package. (For a list of themes please see table 1 on page 9). Default theme used is `theme_minimal`. Labels are added using the command `lab =` which can take either `TRUE` or `FALSE`. If `TRUE` is selected the heatmap shows the correlation coefficients on the graph. Related to it is the `lab_size =` which determines the size of the labels when `lab` is `TRUE`. The `colors = c()` is a vector of three colors which are used to shade from low to mid to high correlation values. I would urge the readers to experiment with the same and decide on the combination which works best for them.

5 Dealing with Missing Data

In statistics, missing data, or missing values, occur when no data value is stored for the variable in an observation. Missing data are a common occurrence and can have a significant effect on the conclusions that can be drawn from the data. Missing data can occur because of nonresponse: no information is provided for one or more items or for a whole unit ("subject"). Some items are more likely to generate a nonresponse than others: for example items about private subjects such as income. Data often are missing in research in economics, sociology, and political science because governments or private entities choose not to, or fail to, report critical statistics, or because the information is not available. Sometimes missing values are caused by the researcher

- for example, when data collection is done improperly or mistakes are made in data entry. Missing values can be treated using following methods :

5.1 Deletion:

The deletion method is used when the probability of missing variable is same for all observations. Here each observation has equal chance of missing value. Deletion can be performed in two types: List Wise Deletion and Pair Wise Deletion.

In **listwise deletion**, we delete observations where any of the variable is missing. Simplicity is one of the major advantage of this method, but this method reduces the power of model because it reduces the sample size. For simplicity we can say that, this method deletes the whole row of observations in which the data is missing. In **pairwise deletion**, we perform analysis with all cases in which the variables of interest are present. Advantage of this method is, it keeps as many cases available for analysis. One of the disadvantage of this method, it uses different sample size for different variables.

5.2 Imputation Methods:

Imputation is a method to fill in the missing values with estimated ones. The objective is to employ known relationships that can be identified in the valid values of the data set to assist in estimating the missing values. Mean / Mode / Median imputation is one of the most frequently used methods. It consists of replacing the missing data for a given attribute by the mean or median (quantitative attribute) or mode (qualitative attribute) of all known values of that variable. It can be of two types. In **Generalized Imputation**, we calculate the mean or median for all non missing values of that variable then replace missing value with mean or median. In **similar case Imputation** In this case, we calculate averages individually of non missing values then replace the missing value based on gender.

5.3 Prediction Model:

Prediction model is one of the sophisticated method for handling missing data. Here, we create a predictive model to estimate values that will substitute the missing data. In this case, we divide our data set into two sets: One set with no missing values for the variable and another one with missing values. First data set become training data set of the model while second data set with missing values is test data set and variable with missing values is treated as target variable. Next, we create a model to predict target variable based on other attributes of the training data set and populate missing values of test data set.We can use regression, ANOVA, Logistic regression and various modeling technique to perform this. There are 2 drawbacks for this approach. Firstly the model estimated values are usually more well-behaved than the true values If there are no relationships with attributes in the data set and the attribute with missing values, then the model will not be precise for estimating missing values.

5.4 KNN Imputation:

In this method of imputation, the missing values of an attribute are imputed using the given number of attributes that are most similar to the attribute whose values are missing. The similarity of two attributes is determined using a distance function. The advantag of KNN is that it can predict both qualitative & quantitative attributes. C of predictive model for each attribute with missing data is not required. Attributes with multiple missing values can be easily treated. Correlation structure of the data is taken into consideration However also have some disadvantages. KNN algorithm is very time-consuming in analyzing large database. It searches through all the dataset looking for the most similar instances. Choice of K-value is

very critical. Higher value of K would include attributes which are significantly different from what we need whereas lower value of K implies missing out of significant attributes.

5.5 Using R to deal with Missing Data

R has various packages to deal with the missing data. We shall discuss the `Hmisc()` package for dealing with missing data. I am not saying that the `Hmisc()` is the best package but it is the simplest to used.

`Hmisc()` (Harrell Jr, with contributions from Charles Dupont, & many others., 2021) is a multiple purpose package useful for data analysis, high – level graphics, imputing missing values, advanced table making, model fitting diagnostics (linear regression, logistic regression cox regression) etc. Amidst, the wide range of functions contained in this package, it offers 2 powerful functions for imputing missing values. These are `impute()` and `aregImpute()`.

`impute()` function simply imputes missing value using user defined statistical method (mean, max, mean). Its default is median. On the other hand, `aregImpute()` allows mean imputation using additive regression, bootstrapping, and predictive mean matching. In bootstrapping, different bootstrap resamples are used for each of multiple imputations. Then, a flexible additive model (non parametric regression method) is fitted on samples taken with replacements from original data and missing values (acts as dependent variable) are predicted using non-missing values (independent variable). Then, it uses predictive mean matching (default) to impute missing values. Predictive mean matching works well for continuous and categorical (binary multi-level) without the need for computing residuals and maximum likelihood fit. The package assumes that there is linearity in the variables being predicted and Fishers optimum scoring method is used for predicting categorical variables.

Instead of creating a random dataset, we shall be using the `msleep()` dataset made available with the `tidyverse()` package. Lets install the package and load it

```
install.packages("Hmisc", dependencies = TRUE)
library(Hmisc)
```

```
## Loading required package:  lattice
## Loading required package:  survival
## Loading required package:  Formula
##
## Attaching package:  'Hmisc'
## The following objects are masked from 'package:dplyr':
##
##     src, summarize
## The following objects are masked from 'package:base':
##
##     format.pval, units
```

Lets now apply the `summary()` function on the dataset

```
summary(msleep)

##      name              genus               vore              order
##  Length:83          Length:83          Length:83          Length:83
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##
##
##
##
##  conservation        sleep_total      sleep_rem      sleep_cycle
##  Length:83          Min.   : 1.90   Min.   :0.100   Min.   :0.1167
```

```
##  Class :character   1st Qu.: 7.85   1st Qu.:0.900   1st Qu.:0.1833
##  Mode  :character   Median :10.10   Median :1.500   Median :0.3333
##                     Mean   :10.43   Mean   :1.875   Mean   :0.4396
##                     3rd Qu.:13.75   3rd Qu.:2.400   3rd Qu.:0.5792
##                     Max.   :19.90   Max.   :6.600   Max.   :1.5000
##                                     NA's   :22      NA's   :51
##      awake          brainwt            bodywt
##  Min.   : 4.10   Min.   :0.00014   Min.   :   0.005
##  1st Qu.:10.25   1st Qu.:0.00290   1st Qu.:   0.174
##  Median :13.90   Median :0.01240   Median :   1.670
##  Mean   :13.57   Mean   :0.28158   Mean   : 166.136
##  3rd Qu.:16.15   3rd Qu.:0.12550   3rd Qu.:  41.750
##  Max.   :22.10   Max.   :5.71200   Max.   :6654.000
##                  NA's   :27
```

From the output we see that `sleep_rem, sleep_cycle` and `brainwt` have 22, 51 and 27 NA's respectively. Lets try and impute the missing values in `sleep_rem` using arithmetic mean:

```
msleep$sleep_rem <- with(msleep, impute(sleep_rem, mean))
summary(msleep$sleep_rem)

##
##  22 values imputed to 1.87541
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.100   1.150   1.875   1.875   2.200   6.600
```

The above command requires a bit of explanation. The LHS tells R that from dataset `msleep` take the variable `sleep_rem` and assign it to the output generated on the RHS. On the RHS it tells the `Hmisc()` to use the dataset `msleep` and fill in all the missing values in the variable `sleep_rem` with mean. The `summary(msleep$sleep_rem)` gives us the summary output for the variable and as can be seen 22 values imputed with the arithmetic mean. Similarly, if we specify `random` then the package replaces all the values with random values. The default replacement value is `median`. We can also use `max` to replace everything with maximum values or with `min` to replace everything with minimum values.

Let us now use a slightly upgraded imputation method. The function `aregImpute()` function, by default, uses predictive mean matching (PMM) method to impute the missing data. One thing to note here is that the original dataset needs to be assigned to a new dataset for analysis purpose. Lets first try the function:

```
msleep.imputed <- aregImpute(~ sleep_total + sleep_rem + sleep_cycle + awake + brainwt +
                               bodywt, data = msleep, n.impute = 10)

## Iteration 1
Iteration 2
Iteration 3
Iteration 4
Iteration 5
Iteration 6
Iteration 7
Iteration 8
Iteration 9
Iteration 10
Iteration 11
Iteration 12
Iteration 13


msleep.imputed
```

```
##
## Multiple Imputation using Bootstrap and PMM
##
## aregImpute(formula = ~sleep_total + sleep_rem + sleep_cycle +
##     awake + brainwt + bodywt, data = msleep, n.impute = 10)
##
## n: 83  p: 6  Imputations: 10   nk: 3
##
## Number of NAs:
## sleep_total   sleep_rem sleep_cycle       awake     brainwt      bodywt
##           0           0          51           0          27           0
##
##             type d.f.
## sleep_total    s    2
## sleep_rem      s    2
## sleep_cycle    s    2
## awake          s    2
## brainwt        s    1
## bodywt         s    2
##
## Transformation of Target Variables Forced to be Linear
##
## R-squares for Predicting Non-Missing Values for Each Variable
## Using Last Imputations of Predictors
## sleep_cycle     brainwt
##       0.843       0.993
```

In the above function, we have created a new variable named as `msleep.imputed`. Then we have instructed the `aregImpute()` function to conduct a predictive mean matching using `sleep_total + awake + bodywt`, and fill in the missing values in the `msleep` dataset. the `n.impute = 5` function tells the function to perform 10 bootstraps on the fitted model. The second line `msleep.imputed` gives the output of the imputation. One major advantage of using `argImpute()` is that it automatically identifies the variable types and treats them accordingly. The R^2 values given provide an indication as to how good the imputation was; obviously the higher the values of R^2 the better the imputation.

Unfortunately, the imputed variables are not so easily available nor are they easily combine able with the old dataset. The data resides in a separately created matrix called `imputed`. So if we want to extract the imputed dataset for analysis we would have to do as follows:

```
new_msleep <- impute.transcan(msleep.imputed, imputation = 1, data = msleep, list.out = TRUE)

##
##
## Imputed missing values with the following frequencies
##  and stored them in variables with their original names:
##
## sleep_cycle     brainwt
##          51          27

summary(new_msleep)

##             Length Class  Mode
## sleep_total 83     -none- numeric
## sleep_rem   83     impute numeric
## sleep_cycle 83     impute numeric
## awake       83     -none- numeric
## brainwt     83     impute numeric
## bodywt      83     -none- numeric
```

Of course, it creates a new dataset; but as you can see from the data, the data sructure is lost. So on a personal level, it is suggested `with()` function be used on all the variables in which there is missing value if the datatset is not too large.

6 Summary

In this session, we have learned about how to use `ggplot2()` for exploratory data analysis. We have also looked at methods of identifying and imputing missing values in the dataset. There are many more functions and packages available. R comes with a base graphics package but the functionality is limited. `ggplot2()` provides easy power into the hands of users to extend the graphics capabilities of R.

7 References

Harrell Jr, F. E., with contributions from Charles Dupont, & many others. (2021). Hmisc: Harrell miscellaneous [Computer software manual]. Retrieved from `https://CRAN.R-project.org/package=Hmisc` (R package version 4.5-0)

Kassambara, A. (2019). ggcorrplot: Visualization of a correlation matrix using 'ggplot2' [Computer software manual]. Retrieved from `https://CRAN.R-project.org/package=ggcorrplot` (R package version 0.1.3)

R Core Team. (2021). R: A language and environment for statistical computing [Computer software manual]. Vienna, Austria. Retrieved from `https://www.R-project.org/`

RStudio Team. (2020). Rstudio: Integrated development environment for r [Computer software manual]. Boston, MA. Retrieved from `http://www.rstudio.com/`

Tukey, J. W. (1997). *Exploratory data analysis*. Addison-Wesley.

Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer-Verlag New York. Retrieved from `https://ggplot2.tidyverse.org`

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., Franois, R., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, *4*(43), 1686. doi: 10.21105/joss.01686